

1. YOUR SKILL PROFILE & GAPS

You already have solid ground in Python, Java, JavaScript, React/Next.js, FastAPI, Docker, Git, and full-stack cybersecurity projects (Argus) plus AI integration (ChoosyAI). This guide deliberately skips basic programming and basic web development.

Highest Priority Gaps (Focus Here)

- TypeScript (strict, real project usage)
- Deeper JavaScript (closures, event loop, this, prototypes)
- Node.js + NestJS backend engineering
- PostgreSQL + relational design + advanced SQL
- OOP principles applied in TypeScript
- Data Structures & Algorithms (interview patterns)
- Software architecture & clean layering
- Testing (unit, integration, E2E)
- AuthN/AuthZ (JWT + RBAC)
- Production code quality, Git workflow, CI/CD
- Basic system design discussion

Secondary (Touch Through Project)

Docker Compose in real app · simple cloud deploy · API docs · validation · logging · error handling · pagination/filtering · optimistic concurrency · observability basics.

2. WHY TEAMFLOW

TeamFlow is a simplified but realistic project-management platform (think Linear/Jira/Trello lite). It forces you to implement the exact skills employers test for junior SWE roles: auth, multi-tenant permissions, relational modeling, business rules, testing, and clean architecture — without cybersecurity or AI features.

Core User Capabilities (Must Finish)

- Register / Login / Logout
- Create workspace, invite/manage members with roles (OWNER / ADMIN / MEMBER)
- Create projects inside a workspace
- Create, assign, update status/priority/deadline of tasks
- Add labels, comments
- Search, filter, paginate, sort tasks
- Activity / history log
- Proper authorization on every sensitive action

■ **Scope control: No microservices, no Redis, no Kafka, no AI features, no fancy real-time. Monolith only.**

3. SKILLS THIS PROJECT WILL TEACH

Everything is taught through implementation, not disconnected theory.

Area	What you practice
TypeScript	Interfaces, types, generics, strict mode, DTOs, narrowing
JavaScript depth	Closures, this, event loop, async/await, prototypes
NestJS	Modules, controllers, services, DI, guards, pipes, filters
PostgreSQL + Prisma	Schema design, relations, migrations, transactions
REST API design	Status codes, validation, pagination, filtering, error shape
Auth	Password hashing, JWT, guards, RBAC, workspace membership checks
Testing	Jest unit, integration, Playwright E2E
Git & CI	Feature branches, meaningful commits, GitHub Actions pipeline
Architecture	Layered monolith, separation of concerns, SOLID in practice
Interview readiness	Explain every major decision + system-design talking points

4. TECHNOLOGY STACK & ARCHITECTURE

Required Stack

Frontend: Next.js (App Router) + React + TypeScript + Tailwind CSS

Backend: Node.js + NestJS + TypeScript + REST

DB: PostgreSQL + Prisma ORM

Testing: Jest + React Testing Library + Playwright

DevOps: Docker + Docker Compose + GitHub Actions

Deploy: Vercel (frontend) + Render/Railway (backend + Postgres)

Architecture (Monolithic Full-Stack)

Frontend (Next.js) → API Client → REST API (NestJS Controllers) → Services (business logic) → Prisma → PostgreSQL

Layer Responsibilities

- **Controllers:** HTTP only — parse request, call service, return response. No business rules.
- **Services:** All business logic, authorization checks, orchestration, transactions.
- **Prisma / Data Access:** Queries and mutations only.
- **DTOs + Validation Pipes:** Shape and validate input at the edge.
- **Guards:** Authentication (JWT) and coarse authorization.
- **Frontend:** UI, form state, loading/error states, calls API client. Never trusts client-side permission checks alone.

Data Flow Example (Create Task)

1. User submits form → 2. Next.js calls POST /projects/:id/tasks with JWT → 3. Guard validates JWT → 4. Validation pipe checks DTO → 5. Controller calls TasksService.create() → 6. Service checks workspace membership + role → 7. Service runs transaction (create task + activity log) → 8. Returns task → 9. Frontend updates UI.

Why this structure?

Separation of concerns, testability (services can be unit-tested without HTTP), clear place for business rules, easy to explain in interviews, and matches how most NestJS codebases are structured.

5. DATABASE DESIGN

Design for correctness and interview discussion, not premature optimization.

Core Entities

- users (id, email, password_hash, name, created_at, updated_at)
- workspaces (id, name, owner_id, created_at, ...)
- workspace_members (workspace_id, user_id, role, joined_at) — composite unique
- projects (id, workspace_id, name, description, status, ...)
- tasks (id, project_id, title, description, status, priority, assignee_id, due_date, version, ...)
- labels (id, workspace_id, name, color)
- task_labels (task_id, label_id) — many-to-many
- comments (id, task_id, author_id, body, created_at)
- activity_logs (id, workspace_id, actor_id, entity_type, entity_id, action, metadata, created_at)

Key Design Decisions (Explain These in Interviews)

- workspace_members is the join table that enables multi-tenancy and RBAC.
- tasks.version supports optimistic concurrency (prevent lost updates).
- Soft-delete only where needed (e.g. projects/tasks) via deleted_at; hard-delete comments if simpler.
- Indexes on foreign keys + common filter columns (status, priority, assignee_id, project_id).
- Timestamps (created_at, updated_at) on almost every table.
- Normalization: labels live at workspace level; task_labels is pure M2M.

Relationships

User 1—N Workspace (as owner) · Workspace 1—N Project · Project 1—N Task · User N—M Workspace (via members) · Task N—M Label · Task 1—N Comment · almost everything → ActivityLog

TIP: Always be able to draw the ER diagram from memory and justify every FK and unique constraint.

6. API DESIGN

Selected Endpoints (REST)

- POST /auth/register · POST /auth/login · POST /auth/logout
- GET/POST /workspaces · GET /workspaces/:id
- POST /workspaces/:id/members · PATCH/DELETE membership
- GET/POST /workspaces/:id/projects · GET/PATCH/DELETE /projects/:id
- GET/POST /projects/:id/tasks · GET/PATCH/DELETE /tasks/:id
- POST /tasks/:id/comments · GET /tasks/:id/comments
- GET /tasks?status=&priority;=&assignee;=&q;=&page;=&limit;=&sort;=
- GET /workspaces/:id/activity

Conventions

- Consistent JSON error shape: { statusCode, message, error, details? }
- DTOs for every write endpoint + class-validator
- Pagination: page + limit (or cursor later). Always return total/count when useful.
- Filtering & sorting via query params; validate enums.
- HTTP status codes: 200, 201, 204, 400, 401, 403, 404, 409, 422

7. AUTHENTICATION & AUTHORIZATION

Authentication = who you are (JWT after login). **Authorization** = what you may do (role + membership checks).

Implementation Outline

- Passwords: bcrypt (or argon2). Never store plaintext.
- Login returns JWT (access token). Prefer httpOnly secure cookie or Authorization: Bearer.
- Global AuthGuard + @Public() decorator for open routes.
- Roles: OWNER, ADMIN, MEMBER stored on workspace_members.
- Every service method that touches a workspace/project/task first verifies membership and required role.

■ **Never trust frontend role checks. Backend must re-validate on every request.**

Business rules examples that live in services: cannot assign task to non-member, cannot modify deleted project, status transitions validated, activity log written inside same transaction.

8. HOW TO USE GROK / AI WITHOUT BECOMING DEPENDENT

You have used AI heavily before. The goal now is understanding, not velocity at the cost of knowledge.

- Attempt the solution yourself first (even a rough version).
- Ask AI for explanations, design options, or reviews — not finished features.
- When AI gives code, rewrite the critical parts in your own words/style.
- After any generated function, force yourself to explain every line out loud or in a note.
- Debug for at least 15–20 minutes before asking AI.
- Never commit code you cannot defend in an interview.
- Use AI as a rubber duck: describe the problem and your current approach first.
- At the end of each day, write 3–5 bullet points: “What I now understand that I didn’t this morning.”

TIP: Interviewers can smell copy-paste. Being able to modify and extend the code live is the real test.

9. WEEK 1 — LEARNING + FOUNDATION (Days 1–7)

✓ **MILESTONE: End of week: Next.js + NestJS + PostgreSQL + Prisma + Docker skeleton with working auth foundation and one protected route.**

DAY 1 — Project Setup, Git, TypeScript Mindset

Learning objectives

Initialize monorepo or separate frontend/backend repos cleanly. Understand why TypeScript strict mode matters. Establish professional Git workflow from day 1.

Exact actions

- Create GitHub repo (or two). Use main + develop or trunk-based with short-lived feature branches.
- Scaffold NestJS app (nest new backend) with TypeScript strict.
- Scaffold Next.js app (create-next-app) with TypeScript + Tailwind + App Router.
- Add .gitignore, .env.example, README skeleton.
- Write first meaningful commit: "chore: initial project structure".

Theory to internalize

TypeScript is a compile-time tool. It disappears at runtime. Strict mode catches entire classes of bugs before they reach production. Interfaces vs type aliases: prefer interface for object shapes that may be extended.

Self-check

Can you explain the difference between interface and type? Why enable strictNullChecks?

Git

Commit message: chore: scaffold backend and frontend with TypeScript strict

DAY 2 — Docker + PostgreSQL + Prisma Foundations

Actions

- Write Dockerfile for NestJS and docker-compose.yml with postgres + backend services.
- Add volume for Postgres data persistence.
- Install Prisma, init, create first User model, run migration.
- Understand migrate vs db push (use migrate for real workflow).

Key concepts

Container vs image. Why we mount volumes. Environment variables via .env and compose. Prisma schema is the source of truth; migrations are versioned SQL.

Deliverable

docker compose up brings up Postgres and backend. Prisma Studio or a simple query works.

DAY 3 — NestJS Core Concepts + First Module

Modules, Controllers, Providers/Services, Dependency Injection. Create AuthModule skeleton. Understand that NestJS is heavily inspired by Angular's DI system.

- Create a health endpoint and a simple UsersService that talks to Prisma.
- Practice constructor injection.

Interview angle

"How does NestJS dependency injection work?" → constructor injection, IoC container, scopes.

DAY 4 — Authentication Foundation

Register + Login endpoints. Hash passwords with bcrypt. Issue JWT. Create JwtAuthGuard. Protect a test route.

Must understand

Why we hash. Difference between authentication and authorization. Where the JWT is validated (guard). Why secret must stay out of source control.

DAY 5 — Database Modeling — Workspaces & Members

Design and migrate Workspace + WorkspaceMember models with roles enum. Unique constraint on (workspace_id, user_id). Practice writing the Prisma schema and understanding the generated SQL.

DAY 6 — Frontend Auth + API Client

Next.js login/register pages. Simple API client (fetch or axios wrapper). Store token safely. Protected route example on frontend (middleware or client check) — but remember backend is source of truth.

DAY 7 — Week 1 Integration + Review

End-to-end: register → login → create workspace → list workspaces. Write a couple of basic unit tests for password hashing / simple service. Clean commits. Update README.

✓ **MILESTONE: Working skeleton: Docker, DB, Auth, Workspace creation.**

10. WEEK 2 — CORE APPLICATION (Days 8–14)

✓ **MILESTONE: End of week: Usable project-management app — workspaces, members, projects, tasks, roles, basic frontend pages.**

DAY 8 — Projects CRUD + Authorization

Project model. Create/list/get/update endpoints. Every operation checks workspace membership and role.

DAY 9 — Tasks — Model & Basic CRUD

Task model with status, priority, assignee, due_date, version. Create and list by project. Validate assignee belongs to workspace.

DAY 10 — Task Updates, Status Transitions, Business Rules

PATCH task. Enforce allowed status transitions if you define any. Generate activity log entry inside a Prisma transaction.

DAY 11 — Members Management + Roles

Invite / change role / remove member. Only OWNER/ADMIN can manage members. Prevent removing the last owner.

DAY 12 — Frontend — Core Pages

Workspace list, project board or list, task detail drawer/page. Forms with validation. Loading and error states. Keep UI clean, not pixel-perfect.

DAY 13 — Labels + Comments (Core)

Labels at workspace level + task_labels. Comments on tasks. Both require membership checks.

DAY 14 — Week 2 Hardening + Manual Test Pass

Fix permission holes. Manual walkthrough of all happy paths and a few negative cases. Meaningful commits. Branch cleanup.

11. WEEK 3 — ENGINEERING DEPTH (Days 15–21)

✓ **MILESTONE: Quality over features: search/filter/pagination, concurrency protection, solid tests, proper error handling.**

DAY 15 — Search, Filter, Pagination, Sorting

GET /tasks with query params. Prisma where + orderBy + skip/take. Indexes on filtered columns. Teach SQL equivalent.

DAY 16 — Optimistic Concurrency

Add version column. On update, check version matches; if not, return 409 Conflict. Explain race conditions and lost updates. This is a strong interview talking point.

DAY 17 — Activity Log + Transactions

Central activity service. Write activity inside the same transaction as the business change. Discuss ACID briefly.

DAY 18 — Validation, Error Handling, Logging

Global exception filter. Consistent error response shape. Structured logging (no secrets). Distinguish expected vs unexpected errors.

DAY 19 — Unit + Integration Tests

Jest for services (mock Prisma). Integration tests for critical endpoints (use test DB or transactions). Aim for meaningful coverage, not 100%.

DAY 20 — E2E Tests with Playwright

One solid happy-path E2E: register → create workspace → project → task → assign → comment → verify. Keep it stable.

DAY 21 — Code Review Pass + Refactors

Apply SOLID/DRY/KISS where it hurts. Extract shared authorization helpers. Clean dead code. Document design decisions.